Banking System - Backend Build Guide

Node.js + Express + MySQL 8 | follows Banking_DB_SQL_Design.pdf | Course: Database Management System

1. The big idea

The database already contains the business logic (13 tables, 8 views, 12 procedures, 1 function, 8 triggers). The Express backend is a thin layer. It does only four things:

- **Authenticate** users (bcrypt password check, JWT token) and know their role.
- **Authorize** each request (is this customer allowed to touch this account?).
- **Call** the right procedure (`CALL sp_...`) or read the right view (`SELECT ... FROM v_...`).
- **Translate** database errors (SIGNAL 45000 messages) into clean HTTP responses.

Never write `UPDATE account SET balance = ...` in backend code. Every money change goes through a procedure, so the transaction, row locks, triggers and audit log always run.

Stack: Node 20+, Express 5 (async errors are forwarded to the error handler automatically), `mysql2` for MySQL, `zod` for validation. Money is kept as **strings** end to end (never JavaScript floats).

2. Prerequisites (database side)

- Run the SQL files in order: `01_schema, 02_indexes, 03_views, 04_procedures, 05_triggers, 06_seed`.
- Every procedure tested in Workbench with a valid case and each failure case (design PDF, section 8).
- Procedure names and parameter lists frozen. The code below depends on that exact parameter order.

Create a limited database user for the app. It can run procedures and read views, but cannot touch tables directly. Procedures run with their definer's rights, so they can still write to tables.

```
CREATE USER 'bank_app'@'localhost' IDENTIFIED BY 'change_me';
GRANT EXECUTE ON BankDB.* TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.v_account_summary     TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.v_customer_accounts   TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.v_loan_status         TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.v_pending_loans       TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.v_branch_summary      TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.v_daily_transactions  TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.v_employee_activity   TO 'bank_app'@'localhost';
-- only for login and the ownership checks:
GRANT SELECT ON BankDB.user_account      TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.customer_account  TO 'bank_app'@'localhost';
GRANT SELECT ON BankDB.loan              TO 'bank_app'@'localhost';
```

3. Project setup

```
bank-backend/
  src/
    index.js           # builds the app, mounts routes, starts the server
    db.js              # pool, callProc(), query(), HttpError, error translation
    auth.js            # bcrypt, JWT, authenticate, requireRole, assertOwnsAccount
    validate.js        # zod helpers: money, id, parse
    routes/
      auth.js          # register, login
      accounts.js      # open, list, joint holder, status, cards
      money.js         # deposit, withdraw, transfer, statement
      loans.js         # apply, pending, decide, pay, emi
      reports.js       # manager reports
  tests/
    concurrency.test.js
  .env
  package.json

npm init -y
npm i express mysql2 bcrypt jsonwebtoken dotenv cors zod
npm i -D nodemon
# package.json: add  "type": "module",  and scripts: { "dev": "nodemon src/index.js" }
```

```
# If bcrypt fails to install on Windows, use:  npm i bcryptjs  and import from 'bcryptjs'

# .env
DB_HOST=localhost
DB_USER=bank_app
DB_PASSWORD=change_me
DB_NAME=BankDB
JWT_SECRET=put-a-long-random-string-here
PORT=8000

# run
npm run dev
```

4. db.js - the only file that talks to MySQL

Two helpers: `callProc` for procedures and `query` for views. The procedures manage their own START TRANSACTION ... COMMIT, so the app just sends the CALL. MySQL error number 1644 means one of your SIGNAL SQLSTATE '45000' messages, and its text is already user-friendly. DECIMAL columns come back as strings (good). `dateStrings: true` keeps dates as plain text so time zones cannot shift them.

```
import mysql from 'mysql2/promise';
import 'dotenv/config';

export const pool = mysql.createPool({
  host: process.env.DB_HOST, user: process.env.DB_USER,
  password: process.env.DB_PASSWORD, database: process.env.DB_NAME,
  connectionLimit: 10,
  dateStrings: true,              // dates stay as 'YYYY-MM-DD' strings
});                               // DECIMAL columns are returned as strings by default

export class HttpError extends Error {
  constructor(status, message) { super(message); this.status = status; }
}

const FORBIDDEN = new Set(['Only a manager can approve loans']);        // -> HTTP 403

function translate(err) {
  if (err.errno === 1644)                                               // our SIGNAL messages
    return new HttpError(FORBIDDEN.has(err.sqlMessage) ? 403 : 400, err.sqlMessage);
  if (err.errno === 1062)                                               // duplicate key
    return new HttpError(409, 'Already exists (CNIC, phone, email or username)');
  console.error(err);
  return new HttpError(500, 'Database error');
}

// name comes from our own code (never from the request), values are bound as parameters
export async function callProc(name, args = []) {
  const marks = args.map(() => '?').join(',');
  try {
    const [res] = await pool.query(`CALL ${name}(${marks})`, args);
    // procedures that SELECT return [rows, okPacket]; others return just an okPacket
    return Array.isArray(res) && Array.isArray(res[0]) ? res[0] : [];
  } catch (e) { throw translate(e); }
}

export async function query(sql, args = []) {
  try { const [rows] = await pool.query(sql, args); return rows; }
  catch (e) { throw translate(e); }
}
```

Pass `null` (not `undefined`) for a missing value such as the online-customer employee_id. An undefined argument makes mysql2 throw.

5. auth.js and validate.js

Roles come from `user_account.role`: customer, teller, manager. The JWT carries role, customerId and employeeId, so every route knows who is calling. Passwords are hashed with bcrypt in the backend; the database only stores the hash.

```
// src/auth.js
import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';
import { HttpError, query } from './db.js';
```

```
export const hashPw  = (p) => bcrypt.hash(p, 10);
export const checkPw = (p, hash) => bcrypt.compare(p, hash);

export const makeToken = (u) => jwt.sign(
  { sub: u.user_id, role: u.role, customerId: u.customer_id, employeeId: u.employee_id },
  process.env.JWT_SECRET, { expiresIn: '8h' });

export function authenticate(req, _res, next) {
  const h = req.headers.authorization || '';
  try {
    req.user = jwt.verify(h.startsWith('Bearer ') ? h.slice(7) : '',
                          process.env.JWT_SECRET, { algorithms: ['HS256'] });
    next();
  } catch { next(new HttpError(401, 'Invalid or expired token')); }
}

export const requireRole = (...roles) => (req, _res, next) =>
  roles.includes(req.user.role) ? next() : next(new HttpError(403, 'Not allowed'));

export async function assertOwnsAccount(user, accountId) {
  if (user.role !== 'customer') return;                // staff can act on any account
  const rows = await query(
    'SELECT 1 FROM customer_account WHERE customer_id=? AND account_id=?',
    [user.customerId, accountId]);
  if (rows.length === 0) throw new HttpError(403, 'This account does not belong to you');
}

// src/validate.js
import { z } from 'zod';
import { HttpError } from './db.js';

// money stays a string like "1500.50"; numbers from JSON are converted to strings first
export const money = z.preprocess(
  (v) => (typeof v === 'number' ? String(v) : v),
  z.string().regex(/^\d{1,13}(\.\d{1,2})?$/, 'Invalid amount')
   .refine((v) => Number(v) > 0, 'Amount must be greater than zero'));

export const dateStr = z.string().regex(/^\d{4}-\d{2}-\d{2}$/, 'Use YYYY-MM-DD');

export function parse(schema, data) {
  const r = schema.safeParse(data);
  if (!r.success) throw new HttpError(400, r.error.issues[0].message);
  return r.data;
}

export function id(v) {
  const n = Number(v);
  if (!Number.isInteger(n) || n <= 0) throw new HttpError(400, 'Invalid id');
  return n;
}
```

Why the ownership check lives in the backend: procedures such as sp_transfer_funds check balances and status, but they do not know *who is logged in*. Without `assertOwnsAccount`, any customer could move money out of someone else's account.

6. Endpoint map (what calls what)

employee_id rule: for staff pass `req.user.employeeId`. For a customer acting online pass `null` (the design says NULL = done online by customer). Never take employeeId from the request body.

Method and path	Calls	Who	Notes
POST /auth/register	sp_register_customer	public	Hash password with bcrypt first, then CALL
POST /auth/login	SELECT user_account	public	bcrypt check, return JWT
POST /accounts	sp_open_account	teller, manager	Fails if deposit < min_balance
GET /accounts	v_account_summary / v_customer_accounts	all	Customer: own only. Staff: search
POST /accounts/:id/joint-holders	sp_add_joint_holder	teller, manager	
PATCH /accounts/:id/status	sp_set_account_status	manager	freeze / unfreeze / close

Method and path	Calls	Who	Notes
POST /accounts/:id/cards	sp_issue_card	teller, manager	Active accounts only
POST /accounts/:id/deposit	sp_deposit	teller, manager	Cash at the counter
POST /accounts/:id/withdraw	sp_withdraw	teller, manager	Enforces min_balance
POST /transfers	sp_transfer_funds	all	Customer must own fromAccount
GET /accounts/:id/statement	sp_get_statement	all	Query: from, to (YYYY-MM-DD)
POST /loans	sp_apply_loan	customer	Rate is set by the server, not the client
GET /loans	v_loan_status	all	Customer: own only
GET /loans/pending	v_pending_loans	manager	Approval queue
POST /loans/:id/decision	sp_decide_loan	manager	Body: { approve: true/false }
POST /loans/:id/payments	sp_pay_installment	customer	Needs fromAccountId the user owns
GET /loans/emi	SELECT fn_calc_emi(...)	all	Calculator, no data change
GET /reports/branch-summary	v_branch_summary	manager	
GET /reports/daily	v_daily_transactions	manager	
GET /reports/employee-activity	v_employee_activity	manager	

7. Writing the routes

Every route has the same shape: validate input, check permission, one `callProc`, return a message. **Parameter order must match the procedure exactly.**

7.1 Register and login (routes/auth.js)

```
import { Router } from 'express';
import { z } from 'zod';
import { callProc, query, HttpError } from '../db.js';
import { hashPw, checkPw, makeToken } from '../auth.js';
import { parse } from '../validate.js';

const router = Router();

const registerSchema = z.object({
  cnic: z.string().regex(/^\d{13}$/, 'CNIC must be 13 digits, no dashes'),
  name: z.string().min(1), phone: z.string().min(1), email: z.string().email(),
  username: z.string().min(3), password: z.string().min(8),
});

router.post('/auth/register', async (req, res) => {
  const b = parse(registerSchema, req.body);
  // rmEmployeeId: null for self-registration (or assign a default RM)
  await callProc('sp_register_customer',
    [b.cnic, b.name, b.phone, b.email, null, b.username, await hashPw(b.password)]);
  res.status(201).json({ message: 'Registered' });
});

router.post('/auth/login', async (req, res) => {
  const b = parse(z.object({ username: z.string(), password: z.string() }), req.body);
  const [u] = await query('SELECT * FROM user_account WHERE username=?', [b.username]);
  if (!u || !(await checkPw(b.password, u.password_hash)))
    throw new HttpError(401, 'Invalid username or password');      // same text for both cases
  res.json({ token: makeToken(u), role: u.role });
});

export default router;
```

7.2 Money routes (routes/money.js)

```
import { Router } from 'express';
import { z } from 'zod';
import { callProc } from '../db.js';
import { authenticate, requireRole, assertOwnsAccount } from '../auth.js';
import { money, dateStr, parse, id } from '../validate.js';

const router = Router();
```

```
const anyUser = [authenticate];
const staff   = [authenticate, requireRole('teller', 'manager')];

router.post('/accounts/:id/deposit', staff, async (req, res) => {
  const { amount } = parse(z.object({ amount: money }), req.body);
  await callProc('sp_deposit', [id(req.params.id), amount, req.user.employeeId ?? null]);
  res.json({ message: 'Deposit successful' });
});

router.post('/accounts/:id/withdraw', staff, async (req, res) => {
  const { amount } = parse(z.object({ amount: money }), req.body);
  await callProc('sp_withdraw', [id(req.params.id), amount, req.user.employeeId ?? null]);
  res.json({ message: 'Withdrawal successful' });
});

router.post('/transfers', anyUser, async (req, res) => {
  const b = parse(z.object({ fromAccount: z.number().int().positive(),
                             toAccount: z.number().int().positive(), amount: money }), req.body);
  await assertOwnsAccount(req.user, b.fromAccount);        // customers: own account only
  await callProc('sp_transfer_funds',
    [b.fromAccount, b.toAccount, b.amount, req.user.employeeId ?? null]);   // null if online
  res.json({ message: 'Transfer successful' });
});

router.get('/accounts/:id/statement', anyUser, async (req, res) => {
  const accountId = id(req.params.id);
  const q = parse(z.object({ from: dateStr, to: dateStr }), req.query);
  await assertOwnsAccount(req.user, accountId);
  res.json(await callProc('sp_get_statement', [accountId, q.from, q.to]));
});

export default router;
```

7.3 Accounts (routes/accounts.js)

```
const openSchema = z.object({ customerId: z.number().int(), branchId: z.number().int(),
                              accountTypeId: z.number().int(), initialDeposit: money });

router.post('/accounts', staff, async (req, res) => {
  const b = parse(openSchema, req.body);
  await callProc('sp_open_account',
    [b.customerId, b.branchId, b.accountTypeId, b.initialDeposit, req.user.employeeId]);
  res.status(201).json({ message: 'Account opened' });
});

router.patch('/accounts/:id/status', [authenticate, requireRole('manager')], async (req, res) => {
  const { status } = parse(z.object({ status: z.enum(['active', 'frozen', 'closed']) }), req.body);
  await callProc('sp_set_account_status', [id(req.params.id), status, req.user.employeeId]);
  res.json({ message: 'Status updated' });
});

router.get('/accounts', anyUser, async (req, res) => {
  if (req.user.role === 'customer')
    return res.json(await query('SELECT * FROM v_customer_accounts WHERE customer_id=?',
                                [req.user.customerId]));
  res.json(await query('SELECT * FROM v_account_summary'));   // staff; add search filters later
});
```

Column names such as `customer_id` must match what you defined in `v_customer_accounts`. Joint holders and cards follow the same pattern (`sp_add_joint_holder(accountId, customerId)` and `sp_issue_card(accountId, cardType)`).

7.4 Loans (routes/loans.js)

```
const LOAN_RATE = 12.5;                 // server decides the rate (config, or by amount/term)
const customerOnly = [authenticate, requireRole('customer')];
const managerOnly  = [authenticate, requireRole('manager')];

// static paths FIRST so '/loans/pending' is never read as '/loans/:id'
router.get('/loans/pending', managerOnly, async (_req, res) => {
  res.json(await query('SELECT * FROM v_pending_loans'));
});

router.get('/loans/emi', anyUser, async (req, res) => {
  const q = parse(z.object({ principal: z.coerce.number().positive(),
                             rate: z.coerce.number().min(0),
```

```
                            months: z.coerce.number().int().positive() }), req.query);
  const [r] = await query('SELECT fn_calc_emi(?,?,?) AS emi', [q.principal, q.rate, q.months]);
  res.json(r);
});

router.post('/loans', customerOnly, async (req, res) => {
  const b = parse(z.object({ branchId: z.number().int(), principal: money,
                             termMonths: z.number().int().positive() }), req.body);
  await callProc('sp_apply_loan',
    [req.user.customerId, b.branchId, b.principal, LOAN_RATE, b.termMonths]);
  res.status(201).json({ message: 'Loan application submitted' });
});

router.post('/loans/:id/decision', managerOnly, async (req, res) => {
  const { approve } = parse(z.object({ approve: z.boolean() }), req.body);
  await callProc('sp_decide_loan', [id(req.params.id), req.user.employeeId, approve]);
  res.json({ message: approve ? 'Approved' : 'Rejected' });
});

router.post('/loans/:id/payments', customerOnly, async (req, res) => {
  const loanId = id(req.params.id);
  const b = parse(z.object({ amount: money, fromAccountId: z.number().int() }), req.body);
  const own = await query('SELECT 1 FROM loan WHERE loan_id=? AND customer_id=?',
                          [loanId, req.user.customerId]);
  if (own.length === 0) throw new HttpError(403, 'This loan does not belong to you');
  await assertOwnsAccount(req.user, b.fromAccountId);
  await callProc('sp_pay_installment', [loanId, b.amount, b.fromAccountId]);
  res.json({ message: 'Installment paid' });
});
```

7.5 index.js

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import { HttpError } from './db.js';
import authRoutes from './routes/auth.js';
import accountRoutes from './routes/accounts.js';
import moneyRoutes from './routes/money.js';
import loanRoutes from './routes/loans.js';
import reportRoutes from './routes/reports.js';

const app = express();
app.use(cors({ origin: 'http://localhost:5173' }));      // Vite dev server
app.use(express.json());

app.get('/health', (_req, res) => res.json({ ok: true }));
app.use(authRoutes, accountRoutes, moneyRoutes, loanRoutes, reportRoutes);
app.use((_req, res) => res.status(404).json({ error: 'Not found' }));

// central error handler: HttpError -> its status, anything else -> 500
app.use((err, _req, res, _next) => {
  if (err instanceof HttpError) return res.status(err.status).json({ error: err.message });
  console.error(err);
  res.status(500).json({ error: 'Server error' });
});

app.listen(process.env.PORT || 8000, () => console.log('API running'));
```

Put *reports.js* in `src/routes/` next to your other routers if you want reports; each one is `SELECT * FROM v_branch_summary` guarded by `requireRole('manager')`.

8. Error handling

The procedures raise short messages. `db.js` turns them into HTTP responses and the frontend shows the `error` field directly to the user.

Database message	HTTP	Typical cause
Insufficient balance	400	Withdraw or transfer larger than allowed
Minimum balance violated	400	Withdraw would go below min_balance, or initial deposit too small
Account is not active	400	Account frozen or closed (trigger also blocks it)
Cannot transfer to the same account	400	fromAccount = toAccount

Database message	HTTP	Typical cause
Loan is not pending	400	Deciding a loan twice
Account balance must be zero to close	400	Closing a funded account
Only a manager can approve loans	403	Non-manager employee_id passed to sp_decide_loan
MySQL 1062 (duplicate)	409	CNIC, phone, email or username already used
Missing or bad token	401	Not logged in or token expired
Wrong role / not your account	403	Blocked by requireRole() or assertOwnsAccount()
Validation failed (zod)	400	Bad amount, CNIC, date or missing field

9. Testing the backend

9.1 Manual

- Use Postman or Thunder Client: POST `/auth/login`, copy the token, set it as a Bearer token for the other requests.
- Test each endpoint with one success and every failure from the table above.
- After each money call, check `transaction` (new row with balance_after) and `audit_log` (balance change logged).
- Export your Postman collection and share it with the frontend team together with the endpoint map.

9.2 Concurrency test (matches your checklist)

Fire opposite transfers at the same time. Expected: no deadlock error, no negative balance, and the total across both accounts unchanged.

```
// tests/concurrency.test.js     run:  TOKEN=<staff or owner token> node tests/concurrency.test.js
const URL = 'http://localhost:8000';
const send = (a, b) => fetch(`${URL}/transfers`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${process.env.TOKEN}` },
  body: JSON.stringify({ fromAccount: a, toAccount: b, amount: '100.00' }),
}).then(async (r) => ({ status: r.status, body: await r.json() }));

const jobs = [];
for (let i = 0; i < 25; i++) jobs.push(send(1, 2), send(2, 1));  // opposite = deadlock bait

const results = await Promise.all(jobs);
console.log(results.filter((r) => r.status === 200).length, 'succeeded');
console.log(results.filter((r) => r.status !== 200).slice(0, 5), '<- failures');
// Then in Workbench:  SELECT SUM(balance) FROM account WHERE account_id IN (1,2);
```

Also run a test where the source account has only enough money for **one** of two simultaneous withdrawals. Exactly one must succeed and the other must return 'Insufficient balance'.

10. Suggested build order

Step	Task	Done when
1	DB files 01-06 loaded, procedures tested in Workbench	Design checklist ticked
2	Project skeleton, db.js, /health	GET /health works and the pool connects
3	auth.js, /auth/register, /auth/login	Token returned, roles enforced
4	Read-only routes from views (accounts, loans, reports)	Every view returns JSON
5	Money routes: deposit, withdraw, transfer, statement	Failure messages match the table
6	Account admin: open, joint holder, status, cards	Each works with the right role only
7	Loan flow: apply, pending, decide, pay, EMI	Full lifecycle ends with loan closed
8	Concurrency tests, error table check	No deadlocks, no negative balance
9	Share endpoint map and Postman collection with the frontend	Frontend can build against it

11. Security checklist

- Bcrypt only. Never store, log or return passwords or hashes.
- App uses the limited `bank_app` user, never root.
- JWT secret in `.env`, which is listed in `.gitignore`.
- Always bind values with ? placeholders. Never build SQL by string concatenation. Procedure names are the only text put in a query, and they come from your code.
- Identity comes from the token (customerId, employeeId), never from the request body.
- Same error text for 'unknown user' and 'wrong password'.
- Restrict CORS to your frontend origin. Consider adding `helmet` and `express-rate-limit` (rate limit login).

12. Things to decide with your group

- Should customers deposit or withdraw online? The guide limits those to tellers and lets everyone transfer; change the `staff` array if you want otherwise.
- Who may freeze or close accounts? The guide uses manager only.
- How loan interest is chosen. The guide uses one server constant; a lookup by amount or term is a simple upgrade.
- How employee login rows are created. Self-registration only makes customers, so tellers and managers need seeded `user_account` rows with bcrypt hashes.
- JSON naming: the guide uses camelCase in request bodies (fromAccount). Agree this with the frontend before they start.